Amrutam Telemedicine Backend: Security

Threat model, OWASP mapping, data classification, encryption and key rotation, and security checklist | Repo: github.com/Ps493/Amrutam-telemedicine-backend

1. Data classification

Class	Data	Controls
Restricted (PHI / PII)	Consultation reason and notes, prescriptions, phone, date of birth, MFA secrets	AES-256-GCM field-level encryption (versioned keys, ciphertext bound to its row via AAD), TLS in transit, disk encryption (KMS), access audited, never logged
Confidential	Email, payment references, audit logs	RBAC, database encryption at rest, hash-chained append-only audit trail
Secrets	JWT secret, encryption keyring, DB credentials	Secrets Manager injected as env vars at runtime; never in repo or image; production start-up fails fast if missing; gitleaks in CI
Public	Doctor name, specialization, fee	Cached and rate-limited

2. Attack surface

Public HTTPS API (auth, search, booking, prescriptions); admin routes; /metrics and /docs (restricted at the network layer); PostgreSQL and Redis (private subnets only); payment gateway egress; CI/CD pipeline and container registry; npm dependencies; operators with production access.

3. STRIDE threat analysis

Threat	Example	Mitigation (implemented and tested unless marked infra)
Spoofing	Credential stuffing, stolen or forged JWT, alg=none	bcrypt (cost 12); lockout after 5 failures for 15 min; auth rate limit; identical error for unknown email vs wrong password with timing equalisation; TOTP MFA, mandatory step-up for doctor and admin operations; HS256 pinned with issuer/audience checks, 15 min access tokens; refresh-token rotation with **reuse detection** that revokes the whole session family
Tampering	Editing another user's record, tampering with audit rows, swapping ciphertexts	Strict zod schemas (unknown fields rejected, which blocks mass assignment); parameterised SQL only; database constraints; GCM auth tags with AAD; audit trigger blocks UPDATE/DELETE; **SHA-256 hash chain** with a verify endpoint detects even privileged tampering
Repudiation	"I never booked / prescribed"	Audit entry written in the **same transaction** as each state change (actor, role, IP, request-id); PHI reads and admin reads are audited
Information disclosure	IDOR, verbose errors, log leakage, stolen DB dump	Ownership checks return 404 (no existence oracle); generic 500 responses; log redaction; field-level encryption makes a dump useless on its own; helmet with HSTS and strict CSP
Denial of service	Request floods, expensive queries, slot hoarding	Global and per-auth rate limits (429, tested); 100 KB body limit; pagination caps; 5 s statement timeout; pool limits; analytics range capped at 366 days and cached; 10 min hold expiry; circuit breaker. Infra: WAF and load balancer
Elevation of privilege	Patient becoming doctor or admin; MFA bypass	Role fixed at registration, admin can never self-register; permission-based RBAC that fails closed; MFA required on privileged routes; doctors must be admin-verified before offering slots

4. OWASP Top 10 (2021) mitigation

Risk	Mitigation in this system
A01 Broken access control	RBAC matrix, object-level ownership checks, 404 for foreign objects, MFA step-up; tests cover IDOR and role escalation
A02 Cryptographic failures	AES-256-GCM field encryption, key versioning and rotation script, bcrypt, TLS 1.2+ with HSTS, KMS-backed storage
A03 Injection	100% parameterised queries; LIKE wildcards escaped; zod validation on every route; SQL-injection payload test
A04 Insecure design	This threat model; saga and idempotency by design; database-level invariants; state machine; rate limits
A05 Security misconfiguration	helmet, no x-powered-by, CSP default-src none, non-root container, read-only root filesystem, dropped capabilities, fail-fast on missing production secrets
A06 Vulnerable components	npm audit gate (fails on high/critical), Trivy container scan, lockfile with npm ci, minimal alpine image; npm removed from the runtime image
A07 Authentication failures	MFA, lockout, password policy (10+ chars, mixed), short-lived tokens, refresh rotation and reuse detection
A08 Software and data integrity	CI-built immutable images, lockfile, hash-chained audit log. Roadmap: image signing (cosign), SBOM
A09 Logging and monitoring failures	Structured logs with request-id, audit trail, alerts on 5xx, latency, backlog and payment failures
A10 SSRF	No user-controlled outbound URLs; egress only to the configured payment provider (network policy in infra)

5. Encryption and key rotation

- **In transit:** TLS 1.2+ at the load balancer; database connections with sslmode=require.

- **At rest:** RDS/EBS/S3 encryption with a KMS key **plus** application-level AES-256-GCM for PHI/PII, which protects against database-level access and leaked backups.
- **Rotation procedure:** add the new key to the keyring (v1,v2) and deploy with ACTIVE_KEY_VERSION=v2, so new writes use v2 while old data stays readable; run `npm run rotate-keys`, which re-encrypts in resumable batches using compare-and-swap; run it with `--check` until it reports 0 stale rows; then retire v1. This was tested: 5 stale values re-encrypted, then 0 remaining.
- **Passwords:** bcrypt cost 12. The KMS master key rotates automatically every year.

6. Audit and compliance

Every state-changing action and every PHI read produces an audit row containing actor, role, action, entity, IP and request-id. Rows are append-only (database trigger), hash-chained, and stored in monthly partitions with WORM archival. `GET /v1/admin/audit-logs/verify` recomputes the chain and reports the first broken row. Admin access to patient data is itself audited.

7. Security checklist

Implemented and tested *unless marked:* ***[infra]*** *= defined in the Terraform/Kubernetes files,* ***[roadmap]*** *= planned.*

Area	Control	Status
Authentication	bcrypt cost 12; password policy; lockout; uniform error plus timing equalisation	Done
	TOTP MFA (secret encrypted); mandatory step-up for doctor/admin; admin cannot self-register	Done
	JWT HS256 pinned, iss/aud verified, 15 min TTL; refresh rotation with reuse detection	Done
	WebAuthn and recovery codes	[roadmap]
Authorization	Permission-based RBAC (fail closed); object-level checks (404 on foreign objects); doctor verification gate	Done
Input and API	zod strict schemas on every route; parameterised SQL; LIKE escaping; body size limit; pagination caps	Done
	Rate limiting (global and auth); Idempotency-Key on all unsafe writes	Done
	helmet (CSP, HSTS, nosniff, frameguard); CORS deny-by-default; no stack traces to clients	Done
Data protection	Field-level AES-256-GCM, AAD-bound, versioned keys; rotation script; log redaction	Done
	KMS/RDS/EBS/S3 encryption; TLS only	[infra]
	Per-record data keys (envelope encryption)	[roadmap]
Integrity and audit	Append-only audit table; SHA-256 hash chain; verify endpoint; PHI read auditing; DB-level invariants	Done
	WORM archive (S3 Object Lock)	[infra]
Supply chain	npm audit gate; Trivy image scan; gitleaks; CodeQL SAST; lockfile with npm ci	Done (in CI)
	Non-root user, read-only filesystem, dropped capabilities, minimal image	Done
	Image signing (cosign) and SBOM	[roadmap]
Operations	Secrets only via env (fail fast in production); health/readiness probes; graceful shutdown; SLO and burn-rate alerts	Done
	Secrets Manager sync; NetworkPolicy; WAF; private subnets	[infra]
	Penetration test; DPDP / HIPAA-style compliance review; consent and data-deletion workflow	[roadmap]

8. Residual risks and future work

Access tokens are stateless, so a suspended user keeps access for up to 15 minutes (mitigation: short deny-list in Redis). HS256 uses a shared secret; move to asymmetric signing with JWKS if multiple services verify tokens. Add MFA recovery codes and WebAuthn, per-record data keys, SIEM export of audit logs, signed webhooks from the real payment provider, and a consent and data-subject deletion workflow.